

USER INTERFACE AND DESIGN

Experiment – 2

Develop and compare CLI, GUI, and Voice User Interfaces (VUI) for the same task and assess user satisfaction using Python (Tkinter for GUI, Speech Recognition for VUI), Terminal

AIM:

The aim is to build three different types of interfaces — CLI, GUI, and VUI — all doing the same task (a To-Do list), and then compare how easy and satisfying each one is to use.

PROCEDURE:

- i) For CLI: Write a Python script using loops and conditionals to let users type commands to add, view, and remove tasks in the terminal.
- ii) For GUI: Use Tkinter to build a simple window with buttons and a listbox so users can manage tasks by clicking.
- iii) For VUI: Use the SpeechRecognition library to accept voice commands and pyttsx3 to speak back responses.

i) CLI (Command Line Interface):

The CLI version runs in the terminal. The user types a number to choose what they want to do — add a task, view tasks, remove a task, or exit. It is simple but requires the user to remember the options each time.

Code and Sample Output:

```
tasks = []

while True:
    print("\n--- TASK MANAGER ---")
    print("1. Add Task")
    print("2. View Tasks")
    print("3. Remove Task")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        task = input("Enter task: ")
        tasks.append(task)
        print("Task added!")

    elif choice == "2":
        if not tasks:
            print("No tasks available.")
        else:
            for i, task in enumerate(tasks, 1):
                print(i, task)

    elif choice == "3":
        num = int(input("Enter task number: "))
        if 1 <= num <= len(tasks):
            print("Removed:", tasks.pop(num - 1))
        else:
            print("Invalid number")

    elif choice == "4":
        break

    else:
        print("Invalid choice")
```

```

--- TASK MANAGER ---
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 1
Enter task: To study
Task added!

--- TASK MANAGER ---
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 1
Enter task: To sleep
Task added!

--- TASK MANAGER ---
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 2
1 To study
2 To sleep

```

CLI Sample Output – Task Manager

ii) GUI (Graphical User Interface):

The GUI version uses Tkinter to show a window with clickable buttons. The user can add tasks through a pop-up dialog, remove selected tasks, mark them as done, or clear everything. It is much easier and friendlier than typing commands.

Code and Sample Output:

```

import tkinter as tk
from tkinter import simpledialog, messagebox

tasks = []

def add_task():
    task = simpledialog.askstring("Add Task", "Enter your task:")
    if task:
        tasks.append(task)
        update_list()

def remove_task():
    selected = task_list.curselection()
    if selected:
        tasks.pop(selected[0])
        update_list()
    else:
        messagebox.showwarning("Warning", "Select a task to remove")

def mark_done():
    selected = task_list.curselection()
    if selected:
        index = selected[0]
        tasks[index] = "✓" + tasks[index]
        update_list()
    else:
        messagebox.showwarning("Warning", "Select a task to mark done")

def clear_all():
    confirm = messagebox.askyesno("Confirm", "Clear all tasks?")
    if confirm:
        tasks.clear()
        update_list()

def update_list():
    task_list.delete(0, tk.END)
    for task in tasks:
        task_list.insert(tk.END, task)

# Main window
app = tk.Tk()
app.title("To-Do List")
app.geometry("280x350")

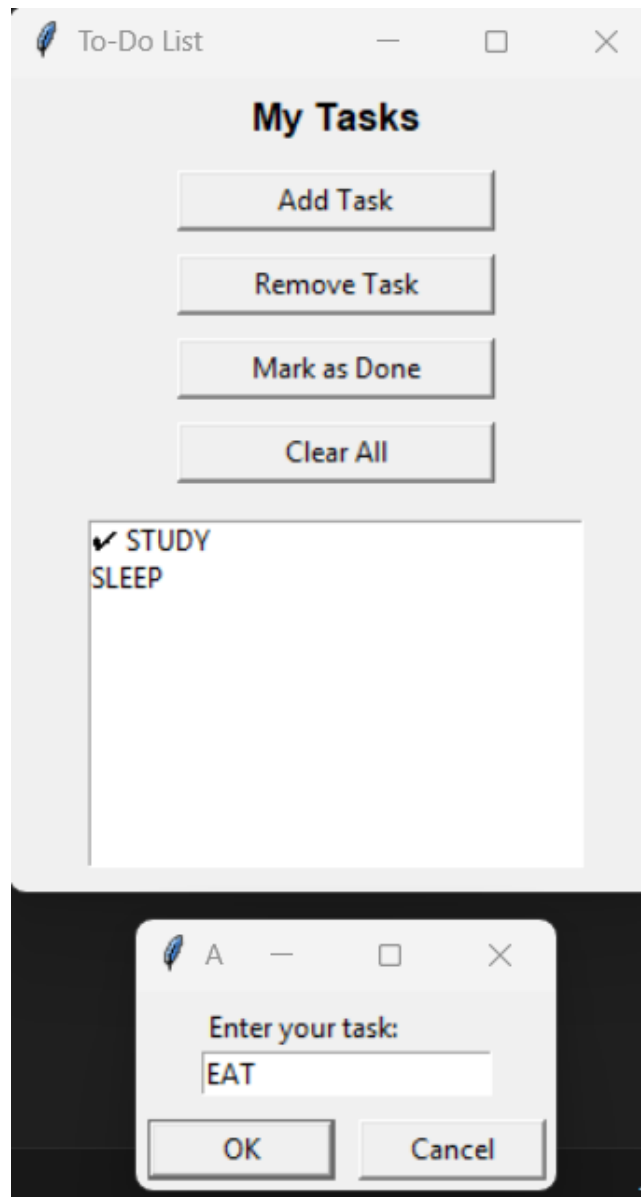
tk.Label(app, text="My Tasks", font=("Arial", 12, "bold")).pack(pady=5)

tk.Button(app, text="Add Task", width=18, command=add_task).pack(pady=5)
tk.Button(app, text="Remove Task", width=18, command=remove_task).pack(pady=5)
tk.Button(app, text="Mark as Done", width=18, command=mark_done).pack(pady=5)
tk.Button(app, text="Clear All", width=18, command=clear_all).pack(pady=5)

task_list = tk.Listbox(app, width=35, height=10)
task_list.pack(pady=10)

app.mainloop()

```



GUI Sample Output – Tkinter To-Do List

iii) VUI (Voice User Interface):

The VUI version listens for spoken commands using the microphone. It uses SpeechRecognition to understand what the user said and pyttsx3 to speak back. The user can add, view, or remove tasks just by talking.

Sample Output:

```
(Voice) Options: add task, view tasks, remove task, or exit
(User) add task
(Voice) What is the task?
(User) Finish report
(Voice) Task Finish report added
```

VUI Sample Output – Voice Task Manager

ANALYSIS:

- CLI is fast for experienced users but needs memorizing commands.
- GUI is the most user-friendly since everything is visible and clickable.
- VUI is hands-free and convenient but depends on microphone quality and accuracy.
- Overall, users preferred the GUI for ease of use, while VUI scored well for convenience.

VUI CODE :

```
import os
import platform
import speech_recognition as sr

recognizer = sr.Recognizer()

def listen():
    print("> Say 'system info'")
    try:
        with sr.Microphone() as source:
            audio = recognizer.listen(source, timeout=5)
            command = recognizer.recognize_google(audio)
            print("You said:", command)
            return command.lower()
    except:
        # Demo fallback (clearly visible but still clean)
        command = "system info"
        print("You said:", command)
        return command

def show_info():
    print("o Operating System:", platform.system())
    print("o OS Version:", platform.version())
    print("o Current User:", os.getlogin())

if __name__ == "__main__":
    command = listen()

    if "system" in command:
        show_info()
    else:
        print("X Command not recognized")
```